

Reinforcement Learning

On-policy Prediction with Approximation: Part I

Min-hwan Oh

Seoul National University

Large-Scale Reinforcement Learning

Reinforcement learning can be used to solve large problems:

- Backgammon: 10^{20} states
- Go: 10^{170} states
- Recommender system: user information, browsing history, etc.
- Self-driving: informal state space (physical world)

Question: How can we scale up our methods for **prediction** (this week) and **control** (next week)?

Value Function Approximation

- Extend tabular methods to problems with arbitrary large state spaces
- Cannot put every state in a table and just record the associated reward
 - ▶ All our state space do not fit in memory.
 - ▶ Even if it did, we don't have the time to fill our table.
- Use a **function approximator** that will take the state in input and output the value of the state
- **Generalization** issue: want our function approximator to generalize the state space
 - ▶ If we get information about one state, can also be useful for similar states.

Value Function Approximation - cont'd

- Approximate v_π from experience generated using a known policy π
- The approximate value function is represented not as a table but as a parametrized function $\hat{v}(s, \mathbf{w})$ with weight vector $\mathbf{w} \in \mathbb{R}^d$.
- We hope $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$.
- $\hat{v}$ might be a linear function in **features** of a state, e.g., a neural network, or decision tree, etc.
- Typically the number of weights d is much smaller than the number of states $|\mathcal{S}|$.

Value Function Updates

- All the prediction methods covered so far involve an update to an estimated value function
- An update ($s \mapsto v$) shifts its value towards a backed-up value (**target**).

Monte Carlo (MC): $S_t \mapsto G_t$

TD(0): $S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$

n -step TD: $S_t \mapsto G_{t:t+n}$

Dynamic Programming (DP): $s \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) | S_t = s]$

Value Function Updates

- All the prediction methods covered so far involve an update to an estimated value function
- An update ($s \mapsto v$) shifts its value towards a backed-up value (target).

Monte Carlo (MC): $S_t \mapsto G_t$

TD(0): $S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$

n -step TD: $S_t \mapsto G_{t:t+n}$

Dynamic Programming (DP): $s \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) | S_t = s]$

- Can interpret each update as an example of the desired input-output behaviour of the value function
 - ▶ (Tabular) estimated value is shifted a fraction of the way towards the update and other states' estimates are left unchanged
 - ▶ Now updating s generalizes so that the estimated value of many other states are changed as well.

The Prediction Objective $\overline{VE}$

Assumptions we made in the tabular setting:

- No need to specify an explicit objective for the prediction because the learned value could come equal with the true value
- An update at one state does not affect any other state

The Prediction Objective $\overline{VE}$

Assumptions we made in the tabular setting:

- No need to specify an explicit objective for the prediction because the learned value could come equal with the true value
- An update at one state does not affect any other state

Now, these two are not statisfied...

- Making one state's estimate more accurate means making others' less accurate
- Need to know which states we care most about with a distribution $\mu(s) \geq 0$ representing how much we care about the error in each state s .
- Error in a state = Mean Squared Error (MSE) between approximate value $\hat{v}(s, \mathbf{w})$ and true value $v_\pi(s)$:

$$\overline{VE}(\mathbf{w}) := \sum_{s \in \mathcal{S}} \mu(s) [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

On-policy Distribution

- Often μ is chosen to be the fraction of time spent in s . **On-policy distribution!**
- In continuing tasks, the on-policy distribution is the stationary distribution under π .
- In episodic tasks, it depends on how the initial states of episodes are chosen.
 - ▶ $h(s)$: probability that an episode begins in each state
 - ▶ $\eta(s)$: the number of time steps spent, on average, in state s in a single episode

$$\eta(s) = h(s) + \sum_{\bar{s}} \eta(\bar{s}) \sum_a \pi(a|\bar{s}) p(s|\bar{s}, a), \forall s \in \mathcal{S}$$

(If discounted, can include γ in the second term)

- The on-policy distribution is then the fraction of time spent in each state normalized to sum to 1:

$$\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')}, \forall s \in \mathcal{S}$$

Performance Objective

$$\overline{VE}(\mathbf{w}) := \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

- Is $\overline{VE}$ a good metric?

Performance Objective

$$\overline{VE}(\mathbf{w}) := \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

- Is $\overline{VE}$ a good metric?
- $\overline{VE}$ is a good starting point but it's not clear that it is the right performance objective
- **Ultimate goal:** find a better (and eventually the best) policy
- Find a global optimum, a weight vector $\mathbf{w}^*$ for which $\overline{VE}(\mathbf{w}^*) \leq \overline{VE}(\mathbf{w}), \forall \mathbf{w}$
- Complex approximators may aim for a local optimum

Stochastic Gradient

- The weight vector is a column vector with a fixed number of real valued components, $\mathbf{w} \doteq (w_1, w_2, \dots, w_d)^T$
- $\hat{v}(s, \mathbf{w})$ is a differentiable function of $\mathbf{w}$ for all s
- Update $\mathbf{w}$ at each time step so $\mathbf{w}_t$ is the weight vector at step t
- In each step, observe a new example $S_t \mapsto v_\pi(S_t)$ assuming for now that we have the true value $v_\pi(S_t)$
- Find the best possible $\mathbf{w}$ by minimizing error on the observed samples

$$\begin{aligned}\mathbf{w}_{t+1} &\doteq \mathbf{w}_t - \frac{1}{2}\alpha \nabla [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)]^2 \\ &= \mathbf{w}_t - \alpha [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)\end{aligned}$$

- $\nabla v(\mathbf{w})$: the partial derivatives w.r.t. components of $\mathbf{w}$

$$\nabla v(\mathbf{w}) \doteq \left(\frac{\partial v(\mathbf{w})}{\partial w_1}, \frac{\partial v(\mathbf{w})}{\partial w_2}, \dots, \frac{\partial v(\mathbf{w})}{\partial w_d} \right)^T$$

Stochastic Gradient Update

$$\begin{aligned}\mathbf{w}_{t+1} &\doteq \mathbf{w}_t - \frac{1}{2}\alpha \nabla [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)]^2 \\ &= \mathbf{w}_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)\end{aligned}$$

- The overall step in $\mathbf{w}_t$ is proportional to the negative gradient of the example's squared error.
- **Stochastic** gradient descent because the update is done on a single example which might have been selected stochastically
- Over many examples, we make many small steps and the overall effect is to minimize an average performance measure (here $\overline{VE}$).
- **Convergence** to a local optimum is guaranteed depending on α

What if we don't have $v_\pi(S_t)$?

- What about when we don't know the true value $v_\pi(S_t)$ but only have an approximation U_t ?

What if we don't have $v_\pi(S_t)$?

- What about when we don't know the true value $v_\pi(S_t)$ but only have an approximation U_t ? Just plug it in place of $v_\pi(S_t)$ in the update!

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

What if we don't have $v_\pi(S_t)$?

- What about when we don't know the true value $v_\pi(S_t)$ but only have an approximation U_t ? Just plug it in place of $v_\pi(S_t)$ in the update!

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

- What can we use for approximations U_t ?

What if we don't have $v_\pi(S_t)$?

- What about when we don't know the true value $v_\pi(S_t)$ but only have an approximation U_t ? Just plug it in place of $v_\pi(S_t)$ in the update!

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

- What can we use for approximations U_t ? Monte Carlo, Bootstrapping, etc.

Monte Carlo Approximation

- If U_t is an unbiased estimate (such as MC), then we have convergence guarantees under certain conditions for α
- $U_t = G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

 Loop for each step of episode, $t = 0, 1, \dots, T - 1$:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$

Bootstrapping Approximation

- n -step TD target: $U_t = G_{t:t+n}$
- DP (like) target: $U_t = \sum_{a,s',r} \pi(a|S_t)p(s',r|s,a)[r + \gamma\hat{v}(s',\mathbf{w}_t)]$
- All biased estimates because they use the current $\mathbf{w}_t$.
- **Semi-gradient** methods: do not converge as robustly as gradient methods but do converge reliably in important cases (e.g., linear case)
- Enables fast learning with online setting

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose $A \sim \pi(\cdot|S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 until S is terminal

Linear Function Approximation

- $\hat{v}(\cdot, \mathbf{w})$ is a linear combination of the weight vector $\mathbf{w}$
- Every state is represented as a vector $\mathbf{x}(s) = [x_1(s), \dots, x_d(s)]^\top \in \mathbb{R}^d$

$$\hat{v}(s, \mathbf{w}) \doteq \mathbf{w}^T \mathbf{x}(s) \doteq \sum_{i=1}^d w_i x_i(s)$$

- $\mathbf{w} = [w_1, \dots, w_d]^\top \in \mathbb{R}^d$ is the weight vector
- Using Stochastic Gradient Descent (SGD) with linear function approximation is easy because the gradient of the approximate value function with respect to $\mathbf{w}$ is:

$$\nabla \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$$

- The linear case SGD update is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \mathbf{x}(S_t)$$

Semi-gradient Convergence of Linear Function Approximation

- Semi-gradient TD(0) also converges to a point near the local optimum under linear function approximation

$$\begin{aligned}\mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha [R_{t+1} + \gamma \mathbf{w}_t^T \mathbf{x}_{t+1} - \mathbf{w}_t^T \mathbf{x}_t] \mathbf{x}_t \\ &= \mathbf{w}_t + \alpha [R_{t+1} \mathbf{x}_t - \mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^T \mathbf{w}_t]\end{aligned}$$

- Shorthand notation: $\mathbf{x}_t = \mathbf{x}(S_t)$
- Once the system has reached steady state, for any given $\mathbf{w}_t$ the expected next weight vector can be written:

$$\mathbb{E}[\mathbf{w}_{t+1} | \mathbf{w}_t] = \mathbf{w}_t + \alpha (\mathbf{b} - \mathbf{A} \mathbf{w}_t)$$

where:

- ▶ $\mathbf{b} \doteq \mathbb{E}[R_{t+1} \mathbf{x}_t] \in \mathbb{R}^d$
- ▶ $\mathbf{A} \doteq \mathbb{E}[\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^T] \in \mathbb{R}^d \times \mathbb{R}^d$

TD Fixed Point

- From the previous slide,
 - ▶ $\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t] \in \mathbb{R}^d$
 - ▶ $\mathbf{A} \doteq \mathbb{E}[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T] \in \mathbb{R}^d \times \mathbb{R}^d$
- In $\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$, it is clear that, if the system converges, it must converge to the weight vector $\mathbf{w}_{TD}$ at which

$$\mathbf{b} - \mathbf{A}\mathbf{w}_{TD} = 0$$

$$\mathbf{b} = \mathbf{A}\mathbf{w}_{TD}$$

$$\mathbf{w}_{TD} \doteq \mathbf{A}^{-1}\mathbf{b}$$

- $\mathbf{w}_{TD}$ is called the **TD fixed point** and the semi-gradient TD(0) converges to this point.
- $\mathbf{A}$ needs to be positive definite to ensure that $\mathbf{A}^{-1}$ exists and to ensure stability (shrinking of $\mathbf{w}$).

Semi-Gradient n -step TD Algorithm

n -step semi-gradient TD for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameters: step size $\alpha > 0$, a positive integer n

Initialize value-function weights $\mathbf{w}$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

All store and access operations (S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take an action according to $\pi(\cdot | S_t)$

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then $T \leftarrow t + 1$

$\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n \hat{v}(S_{\tau+n}, \mathbf{w})$ ($G_{\tau:\tau+n}$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G - \hat{v}(S_\tau, \mathbf{w})] \nabla \hat{v}(S_\tau, \mathbf{w})$

 Until $\tau = T - 1$

Next Class: [Continue] On-policy Prediction with Approximation
(Chapter 9 of Sutton & Barto)